

The AI Engineer Roadmap

From zero code to shipping LLM apps. A topological 6-8 month, day-by-day plan: Python, prompting, RAG, agents, evaluation, and deployment, plus the projects and interview prep that get you hired.

[@rathoreadiitya](#) - Instagram

v1.0 · Updated 2026-06

Inside this sheet

Start here: the student minimum (read this first)

How to use this roadmap (the rules that make it work)

AI Engineer vs ML Engineer (what you are training to be)

What companies actually hire AI engineers to do (2026)

The AI engineer stack (the layers you will learn)

The 8-month map at a glance

--- Part A: the day-by-day plan (the spine) ---

Month 1: Programming + tooling foundation (Weeks 1-4)

Month 2: Data, lean math, ML intuition (Weeks 5-8)

Month 3: How LLMs work + prompt engineering (Weeks 9-12)

Month 4: Building with LLM APIs (Weeks 13-16)

Month 5: Embeddings, vector DBs, RAG (Weeks 17-20)

Month 6: Agents and tools (Weeks 21-24)

Month 7: Evaluation, fine-tuning, safety (Weeks 25-28)

Month 8: Deployment, LLMOps, flagship, interview (Weeks 29-32)

--- Part B: concept reference (read alongside the plan) ---

How an LLM actually works

The prompt engineering pattern bank

Structured generation

Embeddings and vector databases

RAG: the pattern that gets you hired

Agents: LLMs that take actions

Prompt vs RAG vs fine-tune (the decision)

Evaluation and observability (with agent metrics)

Production LLM app architecture

Safety and security (with enterprise concerns)

Cost engineering

Multimodal AI

The data layer behind real AI apps

--- Part C: career assets ---

The tools you'll meet (pick one per layer)

AI-assisted development (coding agents)

The project ladder (your portfolio)

The flagship project: Enterprise Knowledge Assistant

Build vs buy: when to use what

Interview prep: AI engineer questions
Common mistakes that stall beginners
The career reality check
Your hiring timeline
Contributing to open source
What to skip (do not waste months here)
Curated resources (free first)
Final word
Appendix: the jargon decoder
Appendix: the Azure / OpenAI enterprise track

Short on time today? The plan is Part A. Open the current week, do the day, and dip into Part B only for the concept that day touches.

Start here: the student minimum

Honest version. An AI Engineer builds applications on top of existing models (ChatGPT-style models, Claude, Gemini, Llama). You are NOT training models from scratch and you do NOT need a research degree or heavy math. You need to code in Python, understand how an LLM behaves, and be able to build, evaluate, and deploy apps that use prompting, RAG, and agents.

The fresher minimum, five points: (1) Python: write functions, loops, dicts, read JSON, call an API. (2) Mental model of an LLM: it predicts the next token, it has a context window, it can hallucinate. (3) Prompting: roles, few-shot, structured output, chain-of-thought. (4) RAG: retrieve your own documents and feed them to the model with citations. This single skill is what most companies hire for. (5) Ship one real, deployed project with a clean GitHub README. A deployed RAG app beats ten tutorials.

If you only had one month, you would do Month 1 (Python), then jump to Month 3 (prompting) and Month 5 (RAG), and ship one project. The full 8 months exist so you are not faking it: you will understand why each piece works, which is exactly what an interviewer probes.

How to use this roadmap

This is an 8-month plan at roughly 10 to 15 hours a week. Full-time, it compresses to about 6 months. Each week has a goal, a 7-day breakdown, and a deliverable you can show. Day 6 and Day 7 are lighter on purpose: build, review, or rest. Burnout is the most common reason people quit this path, so the light days are not optional.

The five rules that make it work

1. Build more than you watch. For every hour of video, spend two hours typing code. Tutorials feel like progress but only shipped projects prove skill.
2. Topological order is not optional. Every week assumes the weeks before it. Do not jump to agents before you can call an API. The order below is the dependency chain.
3. Ship in public. Push every project to GitHub the day you finish it. Your repo history is your resume.
4. One model, one framework, one vector DB at a time. The tools change constantly; the concepts do not. Learn the concept on one tool, then switching is trivial.
5. Money discipline. Use free tiers and small models while learning. Set a hard monthly spend cap and log your token cost from day one.

The four capstones are the point. By the end you will have shipped: (1) a real LLM app, (2) a chat-with-your-documents RAG app, (3) a task-doing agent with a measured success rate, and (4) a deployed, monitored production service. Those four projects are what get you hired, not the certificates.

AI Engineer vs ML Engineer: what you are training to be

These two roles get confused constantly. Knowing the difference tells you what to study and what to safely skip.

	AI Engineer (this roadmap)	ML Engineer
Core job	Build apps on top of existing models	Train, tune, and serve models
Day to day	Prompting, RAG, agents, APIs, deploy	Data pipelines, training, model tuning
Math needed	Light: intuition only	Heavy: linear algebra, calculus, stats
Main skill	Turn a model into a product	Make a model accurate and efficient
Starts from	Pretrained models you call	Datasets you train on
Hires for	Ship LLM features fast	Move a metric on a model

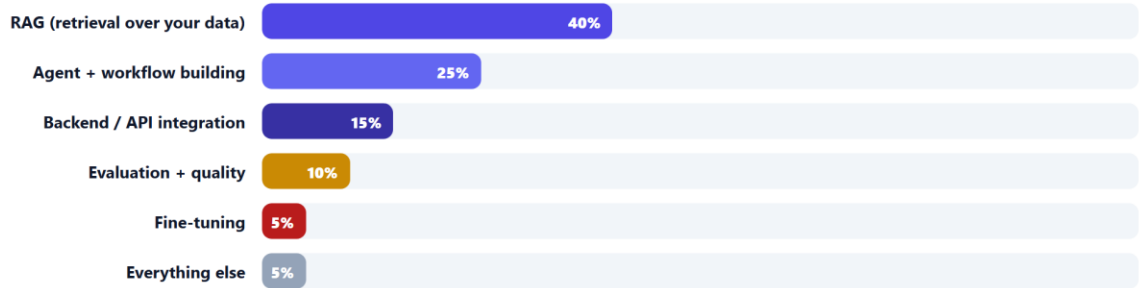
You are training to be the first column. You will still learn enough ML (Month 2) to speak the language and make smart choices, but your job is to ship products, not to publish papers.

What companies actually hire AI engineers to do (2026)

Before the plan, calibrate on the job itself. Students massively overestimate fine-tuning and underestimate plain retrieval and backend work. Here is a rough, directional picture of where the day-to-day work goes. It is not a survey statistic, it is a reality check to aim your study time.

Where an AI engineer's time actually goes

A rough, directional split of real day-to-day work. Not a survey stat, a reality check.



Beginners spend most of their time on fine-tuning. The job is mostly RAG, agents, and plain backend work. Study in that proportion.

Roughly where an AI engineer's time goes. RAG and agents dominate; fine-tuning is tiny.

Slice	Rough share	What it means in practice
RAG	~40%	Put company data into the model: retrieval, chunking, citations, grounding
Agent workflows	~25%	Tool-using agents and multi-step automations with guardrails
Backend / API integration	~15%	Glue: APIs, databases, queues, auth, the plumbing around the model
Evaluation	~10%	Golden sets, scoring, regression tests, quality monitoring
Fine-tuning	~5%	Rare; only after prompting and RAG are not enough
Everything else	~5%	Prompt tuning, cost work, docs, experiments

Read the proportions, then study in the same proportions. The biggest beginner mistake is spending months on fine-tuning and model internals while the actual job is retrieval, agents, and solid backend engineering. Months 4, 5, and 6 of this plan are weighted to match.

The AI engineer stack

Everything in this roadmap fits into a stack. Lower layers support the ones above. You learn bottom to top, which is exactly the month order.

The AI engineer stack

You learn bottom to top. Higher layers depend on the ones below.



The layers of an AI engineering project, bottom to top.

A real app uses several layers at once: Python glue code, a model behind an API, a prompt layer, a retrieval layer (RAG), maybe an agent loop, an evaluation harness, and a deployment wrapper. When something breaks in production, knowing which layer it lives in is half the fix.

The 8-month map at a glance

The 8-month map

A dependency chain. Each month sets up the next. Compresses to ~6 months full-time.

MONTH 1	Python + tooling Code, Git, command line, call your first API.
MONTH 2	Data + lean math + ML intuition NumPy/Pandas, just-enough math, what training and embeddings are.
MONTH 3	How LLMs work + prompt engineering Next-token model, decoding controls, the prompt pattern bank.
MONTH 4	Building with LLM APIs Messages, structured output, function calling. Ship a chat app. Capstone 1.
MONTH 5	Embeddings + vector DBs + RAG The pattern that gets you hired. Chat with your documents. Capstone 2.
MONTH 6	Agents + tools ReAct loop, frameworks, MCP, guardrails. An agent that does a task. Capstone 3.
MONTH 7	Evaluation + fine-tuning + safety Measure quality, fine-tune once, defend against prompt injection.
MONTH 8	Deployment + LLMOps + career Production service, monitoring, portfolio, interviews. Capstone 4.

The dependency chain. Each phase sets up the next.

Month	Phase	What you can do by the end
1	Python + tooling	Code, use Git, call an API, ship a script
2	Data + lean ML	Handle data, understand training and embeddings
3	LLMs + prompting	Explain how LLMs work, write strong prompts
4	Building with APIs	Ship a deployed LLM app with a UI
5	Embeddings + RAG	Build chat-with-your-documents over real data
6	Agents + tools	Build an agent that completes a real task
7	Eval + fine-tune + safety	Measure quality, fine-tune, defend the app
8	Deploy + LLMOps + career	Run it in production, land interviews

Why this order is fixed: you cannot prompt before you can code, cannot build RAG before you understand embeddings, cannot build agents before you can do function calling, and cannot deploy before you have something worth deploying. Follow the chain.

Part A: the day-by-day plan

This is the spine. Eight months, 32 weeks, every day mapped. Open the current week each day and do the row. The concept deep-dives you will need are in Part B.

Month 1: Programming + tooling foundation

You cannot build AI apps without code. This month you go from zero to a confident Python beginner who can use Git, call an API, and ship a small script. Do not skip it even if you are impatient to touch AI. Every later week assumes you can write a function, read JSON, and push to GitHub.

Week 1: Python setup and the absolute basics

Goal: Install everything and write your first 50 lines of Python.

Day	Focus	Do exactly this
Day 1	Setup	Install Python + VS Code. Run a hello-world script. Learn interpreter vs script file.
Day 2	Variables + types	int, float, str, bool. print(), input(), f-strings, type().
Day 3	Operators	Arithmetic, comparison, and/or/not. Integer vs float division.
Day 4	Strings	Indexing, slicing, upper/lower/split/replace/strip/join.
Day 5	Conditionals	if / elif / else, truthiness, nesting.
Day 6	Practice	FizzBuzz, even/odd, max of three, simple calculator. 10 tiny problems.
Day 7	Review	Rewrite Day 6 solutions cleanly. Light day.

Week deliverable: A folder of 10 clean tiny Python scripts that run without errors.

Week 2: Data structures and loops

Goal: Store and loop over data the way every program does.

Day	Focus	Do exactly this
Day 1	Lists	Create, index, append, slice, iterate.
Day 2	Loops	for + range(), while, break, continue.
Day 3	Dictionaries	Keys/values, get(), iterate, when to reach for a dict.
Day 4	Sets + tuples	Membership tests, uniqueness, immutability.
Day 5	Comprehensions	List/dict comprehensions, enumerate, zip.
Day 6	Practice	Word-frequency counter, group items, remove duplicates.
Day 7	Review	Self-quiz on all four structures. Light day.

Week deliverable: A word-frequency counter that reads a text file and prints the top 10 words.

Week 3: Functions, files, classes, environments

Goal: Write reusable code and manage a Python project properly.

Day	Focus	Do exactly this
Day 1	Functions	def, parameters, return, default args, *args/**kwargs.
Day 2	Modules	Scope, docstrings, import math/random/datetime.

Day	Focus	Do exactly this
Day 3	Exceptions	try/except/finally, raise, reading a traceback.
Day 4	Files + JSON	open/read/write, with-statement, json.load/dump.
Day 5	Classes	__init__, attributes, methods. Write one simple class.
Day 6	Environments	venv, pip install, requirements.txt.
Day 7	Build	A CLI to-do app that saves tasks to a JSON file.

Week deliverable: A working command-line to-do app backed by a JSON file.

Week 4: Git, the command line, and your first API call

Goal: Ship code to GitHub and pull data from the internet.

Day	Focus	Do exactly this
Day 1	Command line	cd, ls, mkdir, file paths, running scripts from terminal.
Day 2	Git	init, add, commit, log, .gitignore.
Day 3	GitHub	Create a repo, push, write a README, clone.
Day 4	How the web works	HTTP requests/responses, status codes, what JSON is.
Day 5	requests	GET a public API, parse the JSON response.
Day 6	Secrets	API keys, .env files, os.environ. Never commit a key.
Day 7	Project 1	A script that calls a public API and saves results. Push to GitHub.

Week deliverable: Project 1 on GitHub: a script that fetches and stores data from a public API.

Month 2: Data, lean math, and ML intuition

An AI engineer is not an ML researcher, so you do not need months of math. You DO need enough intuition to not be lost: what a vector is, what training means, why models overfit, and how text becomes numbers. This month gives you that floor and nothing extra. If a topic feels too mathematical, stay at the intuition level and move on.

Week 5: Notebooks, NumPy, Pandas

Goal: Handle data like a working engineer.

Day	Focus	Do exactly this
Day 1	Notebooks	Jupyter / Google Colab, cells, markdown.
Day 2	NumPy	Arrays, shapes, vectorized operations.
Day 3	Pandas I	Series, DataFrame, read_csv, head/info/describe.
Day 4	Pandas II	Filter, select columns, groupby, sort.
Day 5	Plots	matplotlib histogram, line, bar.
Day 6	Mini EDA	Explore one Kaggle CSV end to end.
Day 7	Review	Light day.

Week deliverable: A Colab notebook with a small exploratory analysis of a real dataset.

Week 6: The math you actually need (lean)

Goal: Build intuition for the few ideas that recur in AI.

Day	Focus	Do exactly this
Day 1	Vectors	What a dot product means: similarity. This underlies embeddings.
Day 2	Matrices	Data as rows (examples) and columns (features).
Day 3	Probability	Events, conditional probability, distributions (intuition).
Day 4	Statistics	Mean, variance, standard deviation, normalization.
Day 5	Gradient	Slope downhill intuition. No heavy calculus.
Day 6	Watch	3Blue1Brown linear algebra + neural network essence videos.
Day 7	Review	Write a one-page cheat note. Light day.

Week deliverable: A one-page note in your own words explaining vectors, similarity, and gradients.

Week 7: Machine learning intuition + scikit-learn

Goal: Train one real model and understand what happened.

Day	Focus	Do exactly this
Day 1	What ML is	Learning a function from data. Supervised vs unsupervised.
Day 2	Splits	Train / validation / test. Why. Data leakage.
Day 3	Model = parameters	Features and labels. A model is numbers fit to data.
Day 4	Over/underfitting	Bias-variance intuition. Why more is not always better.
Day 5	Train one	scikit-learn classifier on iris or titanic, end to end.
Day 6	Metrics	Accuracy, precision, recall, confusion matrix.
Day 7	Review	Light day.

Week deliverable: A notebook that trains, evaluates, and explains one scikit-learn classifier.

Week 8: From text to numbers (NLP + neural-net intuition)

Goal: Understand the bridge from language to vectors that LLMs are built on.

Day	Focus	Do exactly this
Day 1	Why numbers	Models only do math, so text must become numbers.
Day 2	Tokenization	Words, subwords, tokens. Why tokens not characters.
Day 3	Embeddings	Meaning as a vector. Similar meanings sit close together.
Day 4	Neural nets	Layers, weights, activations (intuition only).
Day 5	Training	Loss and gradient descent: the model nudges weights to be less wrong.
Day 6	Play	Cosine similarity by hand on a few embedding vectors.
Day 7	Review	You now have the foundation. Light day.

Week deliverable: A short script that embeds 5 sentences and ranks them by similarity to a query.

Month 3: How LLMs work + prompt engineering

Now the AI part begins. First you build a correct mental model of what an LLM is (a next-token predictor, not a database and not a brain). Then you learn prompting, the skill that pays off most in this whole field: most real improvements come from a better prompt long before anyone fine-tunes anything.

Week 9: Inside an LLM (intuition, not math)

Goal: Know exactly what the model is doing when it answers.

Day	Focus	Do exactly this
Day 1	Next token	An LLM predicts the next token, over and over.
Day 2	Context window	Tokens in + out. Long inputs cost more and can overflow.
Day 3	Attention	Transformers 'look at' the relevant earlier words (intuition).
Day 4	Train vs infer	Pretraining, fine-tuning, and inference: three different things.
Day 5	Decoding	temperature, top-p, top-k, max tokens: how randomness is controlled.
Day 6	Hallucination	Why a confident wrong answer happens, and what reduces it.
Day 7	Review	Light day.

Week deliverable: A note explaining, to a friend, why an LLM hallucinates and how temperature works.

Week 10: Picking a model

Goal: Know which model to reach for and why.

Day	Focus	Do exactly this
Day 1	Closed models	OpenAI (GPT / o-series), Anthropic Claude, Google Gemini.
Day 2	Open models	Llama, Mistral, Qwen, Gemma. Where they run.
Day 3	Open vs closed	Cost, privacy, control, and quality tradeoffs.
Day 4	Hugging Face	The Hub, model cards, licenses.
Day 5	Run local	Ollama or LM Studio: run a small open model on your machine.
Day 6	SLMs	Small language models: when smaller and cheaper wins.
Day 7	Decide	Pick your default model + a backup. Light day.

Week deliverable: A one-page comparison of 3 models you could use, with when you would pick each.

Week 11: Prompt engineering fundamentals

Goal: Write prompts that work on the first try more often.

Day	Focus	Do exactly this
Day 1	Prompt anatomy	Role, instruction, context, examples, output format.

Day	Focus	Do exactly this
Day 2	Few-shot	Zero-shot vs few-shot. Show examples to steer the model.
Day 3	Be specific	Constraints, delimiters, exact output schema.
Day 4	Message roles	system vs user vs assistant. What the system prompt is for.
Day 5	Structured output	Ask for JSON / lists / tables you can parse.
Day 6	Iterate	Run one prompt on 10 inputs, find failure modes, fix them.
Day 7	Build	A reusable prompt-template library. Light day.

Week deliverable: A small library of 5 tested, reusable prompt templates.

Week 12: Advanced prompting

Goal: Get reliable reasoning and ship a prompt-only tool.

Day	Focus	Do exactly this
Day 1	Chain-of-thought	Ask the model to think step by step for hard tasks.
Day 2	ReAct	Reason then act. This is the seed of agents you build later.
Day 3	Chaining	Self-consistency, prompt chaining, breaking a task into steps.
Day 4	Persona + tone	Role prompting and controlling voice.
Day 5	Injection intro	Why user input is dangerous inside a prompt.
Day 6	Failure patterns	Common prompt failures and their fixes.
Day 7	Project 2	A prompt-only tool (summarizer / rewriter / classifier). Push to GitHub.

Week deliverable: Project 2 on GitHub: a useful tool built with prompting alone (no extra ML).

Month 4: Building with LLM APIs

Time to write code that talks to models. You learn the API surface every provider shares (messages, system prompt, streaming, usage), then structured output and function calling, then you wrap it all in a real app with a UI and ship it. By the end of this month you have your first deployed AI product.

Week 13: Your first LLM API calls

Goal: Call a model from Python and control the output.

Day	Focus	Do exactly this
Day 1	Keys	Get an API key (or use a free tier). Set up .env.
Day 2	First call	Send a message, read the response.
Day 3	Messages array	System prompt + conversation history.
Day 4	Streaming	Stream tokens. Read the usage/token counts in the response.
Day 5	Controls	Set temperature and max tokens in code.
Day 6	Errors	Handle rate limits, timeouts, and retries.
Day 7	Review	Track your token spend. Light day.

Week deliverable: A solid Python wrapper that calls an LLM with retries and logs token usage.

Week 14: Structured output and function calling

Goal: Make the model return data you can use and call your code.

Day	Focus	Do exactly this
Day 1	JSON output	Structured outputs. Validate with pydantic.
Day 2	Tool calling	Let the model decide to call one of your functions.
Day 3	Run the tool	Validate the call (function exists, args sane), run it, feed the result back.
Day 4	Cost control	Token budgets, truncation, caching responses.
Day 5	Config prompts	Move prompts out of code into templates/config.
Day 6	Logging	Log every prompt + response for debugging.
Day 7	Review	Light day.

Week deliverable: A script where the model calls a real function (e.g. weather or calculator) and uses the result.

Week 15: Build a chat app + frameworks

Goal: Ship a chat UI and learn when a framework helps.

Day	Focus	Do exactly this
Day 1	UI	Streamlit or Gradio: a UI in pure Python.
Day 2	Wire it up	Connect the LLM to the chat UI.
Day 3	Memory	Conversation history. Trim it to fit the context window.
Day 4	Frameworks	What LangChain / LlamaIndex do, and when to skip them.
Day 5	Guardrail	Add a system persona and a basic safety instruction.
Day 6	Deploy	Hugging Face Spaces or Streamlit Cloud.
Day 7	Polish	Write the README. Light day.

Week deliverable: A deployed chat app anyone can open in a browser.

Week 16: Capstone 1: a real LLM app

Goal: Ship something a real person would actually use.

Day	Focus	Do exactly this
Day 1	Pick	A real use case: study buddy, email writer, support bot.
Day 2	Design	The prompts and the flow.
Day 3	Core	Build the main loop.
Day 4	UI + memory	Add the interface and conversation memory.
Day 5	Golden set	Save 12-15 real inputs with pass/fail checks. Re-run them after every change (regression test). This is eval, you go deep on it in Month 7.

Day	Focus	Do exactly this
Day 6	Deploy	Ship it and write a demo post.
Day 7	Showcase	GitHub repo with screenshots. Light day.

Week deliverable: Capstone 1: a deployed, documented LLM app on your GitHub.

Month 5: Embeddings, vector databases, and RAG

This is the single most important month for employability. Retrieval-augmented generation (RAG) is how almost every real company puts its own data into an LLM app. You build up from embeddings to vector search to a full RAG pipeline, then make it good enough to ship.

Week 17: Embeddings for real

Goal: Turn documents into searchable meaning.

Day	Focus	Do exactly this
Day 1	Recap	Embeddings: text becomes a vector of meaning.
Day 2	Generate	Make embeddings via an API or sentence-transformers.
Day 3	Similarity	Cosine similarity. Build semantic search by hand.
Day 4	Chunking	Split documents. Chunk size and overlap matter.
Day 5	Build	Semantic search over your own notes.
Day 6	Choices	Embedding model choice and cost.
Day 7	Review	Light day.

Week deliverable: A semantic search tool: type a question, get the most relevant chunks of your notes.

Week 18: Vector databases

Goal: Store and query vectors at real scale.

Day	Focus	Do exactly this
Day 1	Why a vector DB	Fast nearest-neighbor search over millions of vectors.
Day 2	Local DB	Chroma or FAISS: add and query vectors.
Day 3	Filtering	Metadata filters and hybrid (keyword + vector) search.
Day 4	Managed	Pinecone, Qdrant, Weaviate, pgvector: the main options.
Day 5	Indexing	Recall vs speed tradeoffs (intuition).
Day 6	Load	Put a real dataset into a vector DB.
Day 7	Review	Light day.

Week deliverable: A vector DB loaded with a real document set you can query semantically.

Week 19: RAG (retrieval-augmented generation)

Goal: Make an LLM answer using your documents, with sources.

Day	Focus	Do exactly this
Day 1	The idea	Retrieve relevant chunks, put them in the prompt, then answer.
Day 2	Pipeline	load, chunk, embed, store, retrieve, generate.
Day 3	Build	A minimal RAG over a PDF or a website.
Day 4	Citations	Show which source each answer came from.
Day 5	RAG vs fine-tune	Why RAG wins for fresh or private knowledge.
Day 6	Failure modes	Bad chunks and wrong retrieval, and how to spot them.
Day 7	Design	Sketch the architecture for a support chatbot and a PDF assistant on paper. Think in boxes and arrows.

Week deliverable: A working RAG that answers questions about a document and cites its sources.

Week 20: Better RAG + Capstone 2

Goal: Upgrade retrieval quality and ship a documents chatbot.

Day	Focus	Do exactly this
Day 1	Re-ranking	Reorder retrieved chunks so the best ones win.
Day 2	Query rewriting	Rephrase or multi-query the user's question.
Day 3	Eval RAG	Is the answer actually grounded in the sources?
Day 4	Build	Capstone 2: chat with your documents.
Day 5	Polish	Citations + a clean UI.
Day 6	Deploy	Ship it.
Day 7	Showcase	GitHub repo + demo. Light day.

Week deliverable: Capstone 2: a deployed 'chat with your documents' RAG app.

Month 6: Agents and tools

Agents are LLMs that can take actions: call tools, search, run code, loop until done. They are the most hyped and most fragile part of the field, so you learn to build them carefully with stopping conditions, tracing, and guardrails. You build the loop by hand first, then with a framework.

Week 21: Tool use and the agent loop

Goal: Build an agent from scratch so you understand it.

Day	Focus	Do exactly this
Day 1	Tools	Recap function calling. Tools are the agent's hands.
Day 2	ReAct loop	think, act, observe, repeat.
Day 3	Single tool	An agent with one tool (calculator or web search).
Day 4	Routing	Multiple tools. How the agent picks one.
Day 5	Stop conditions	Max steps, loop and cost control.

Day	Focus	Do exactly this
Day 6	Tracing	Print and inspect every step the agent takes.
Day 7	Review	Light day.

Week deliverable: A from-scratch agent that solves multi-step tasks with one or two tools.

Week 22: Agent frameworks and MCP

Goal: Use a real framework, then learn the tool standard the field is moving to.

Day	Focus	Do exactly this
Day 1	Frameworks	Pick one (LangGraph, smolagents, CrewAI, OpenAI Agents SDK) and rebuild your Week 21 agent in it.
Day 2	MCP fundamentals	What Model Context Protocol is and why a shared standard for tools and data matters.
Day 3	MCP clients	Connect your agent as a client to an existing MCP server.
Day 4	MCP servers	Run community servers (files, web, GitHub) and call their tools from your agent.
Day 5	Build a server	Write a small custom MCP server that exposes one of your own tools.
Day 6	MCP security	Untrusted servers, tool permissions, least privilege. Never auto-trust a server.
Day 7	Review	Light day.

Week deliverable: Your agent running in a framework, plus one custom MCP server you built and connected.

Week 23: Real agents

Goal: Make an agent reliable enough to trust with a task.

Day	Focus	Do exactly this
Day 1	Planning	Task decomposition: break a goal into steps.
Day 2	Memory	Short-term vs long-term agent memory.
Day 3	Multi-agent	Roles that collaborate. When it helps and when it hurts.
Day 4	Guardrails	Validation, allow-lists, human-in-the-loop.
Day 5	Reliability	Cost, latency, and failure handling.
Day 6	Design	Scope your capstone agent.
Day 7	Review	Light day.

Week deliverable: A design doc + prototype for a guard-railed agent that does a real task.

Week 24: Capstone 3: an agent that does a task

Goal: Ship an agent and measure how often it succeeds.

Day	Focus	Do exactly this
Day 1	Scope	Research assistant, data-to-report, or inbox triage.
Day 2	Tools	Define the tools and the flow.
Day 3	Build	The core agent.
Day 4	Safety	Add guardrails and tracing.
Day 5	Measure	Test on real tasks. Record a success rate.
Day 6	Deploy	Ship and demo.
Day 7	Showcase	GitHub repo with the success metric. Light day.

Week deliverable: Capstone 3: a deployed agent with a measured success rate in its README.

Month 7: Evaluation, fine-tuning, and safety

This month separates hobbyists from engineers. You learn to measure quality instead of eyeballing it, to decide when fine-tuning is even worth it (usually it is not, do this after prompting and RAG), and to defend your app against the attacks every LLM product faces.

Week 25: Evaluation and observability

Goal: Replace 'it looks fine' with a number you can track.

Day	Focus	Do exactly this
Day 1	Why eval	'Looks good' is not a metric. You cannot improve what you do not measure.
Day 2	Eval set	Build a set of inputs + expected behavior.
Day 3	LLM-as-judge	Score outputs with a model + a rubric.
Day 4	Tracing	Observability: LangSmith, Langfuse, or Phoenix.
Day 5	Track	Latency, cost, and failure rate.
Day 6	Regression	Re-run evals before every prompt change.
Day 7	Apply	Add evals to a past capstone. Light day.

Week deliverable: An eval suite that scores one of your apps and catches regressions.

Week 26: Prompt vs RAG vs fine-tune

Goal: Know which lever to pull, in the right order.

Day	Focus	Do exactly this
Day 1	The order	Try prompting first, then RAG, then fine-tuning.
Day 2	When to fine-tune	What fine-tuning is good at (style, format, narrow tasks).
Day 3	LoRA / PEFT	Cheap fine-tuning intuition.
Day 4	Dataset	Building a fine-tune dataset: format and quality.
Day 5	Costs + risks	Compute, staleness, and overfitting.
Day 6	When not to	Cases where fine-tuning is the wrong tool.
Day 7	Review	Light day.

Week deliverable: A decision note: for 3 scenarios, say whether to prompt, RAG, or fine-tune, and why.

Week 27: Hands-on light fine-tuning

Goal: Fine-tune once so you actually understand it.

Day	Focus	Do exactly this
Day 1	Pick a task	Something the base model gets wrong (a tone or strict format).
Day 2	Dataset	Prepare a small, clean dataset.
Day 3	Fine-tune	OpenAI fine-tune API or LoRA on a small open model.
Day 4	Evaluate	Compare against the base model on your eval set.
Day 5	Use it	Deploy or call the fine-tuned model.
Day 6	Write up	Document what changed and by how much.
Day 7	Review	Light day.

Week deliverable: A fine-tuned model + a before/after comparison on your own eval set.

Week 28: Safety and security

Goal: Defend your app against the attacks every LLM product faces.

Day	Focus	Do exactly this
Day 1	Prompt injection	The number one LLM vulnerability. Jailbreaks.
Day 2	Defenses	Input/output filtering, allow-lists, least privilege for tools.
Day 3	Privacy	PII handling. Do not leak user data into logs or prompts.
Day 4	Moderation	Content moderation APIs.
Day 5	Grounding	Reduce hallucination with citations and a refusal path.
Day 6	Responsible AI	Bias, transparency, and stating the system's limits.
Day 7	Apply	Add a safety layer to a capstone. Light day.

Week deliverable: A safety layer (input filtering + output checks) added to one of your apps.

Month 8: Deployment, LLMOps, portfolio, and interview

The final month makes you hireable. You take your apps from notebooks to real services, learn how teams operate LLM apps in production, polish a portfolio that proves you can ship, and prepare for AI engineer interviews. The centerpiece is one flagship project, the Enterprise Knowledge Assistant, that pulls your RAG, agent, eval, and deployment skills into a single resume-defining build. Finish this month and you can apply with confidence.

Week 29: Flagship build: Enterprise Knowledge Assistant (part 1)

Goal: Start the one project that combines everything you have learned.

Day	Focus	Do exactly this
Day 1	Scope	Define the assistant: RAG over a document set, an agent with 2-3 tools, auth, the lot.
Day 2	RAG core	Wire in your best retrieval pipeline with citations.
Day 3	Agent layer	Add an agent that can use tools (search, lookups) on top of retrieval.
Day 4	API	Wrap it in a FastAPI service with auth and rate limiting.
Day 5	Caching	Add response + semantic caching to cut cost and latency.
Day 6	Docker	Containerize the service (Docker basics).
Day 7	Review	Light day.

Week deliverable: The flagship assistant running locally: RAG + agent + auth + caching behind one API.

Week 30: Flagship build: deploy + LLMOps (part 2)

Goal: Ship the flagship to production and operate it like a team would.

Day	Focus	Do exactly this
Day 1	Deploy	Push to Render, Railway, Azure, or an AWS free tier.
Day 2	Monitor	Logging, tracing, and a cost dashboard in production.
Day 3	Eval gate	Run your golden-set evals on every change (CI/CD for AI).
Day 4	Versioning	Version prompts and configs like code.
Day 5	Drift	Watch quality and alert when it drops. Add budget guards.
Day 6	Architecture	Draw the full system diagram for the README.
Day 7	Review	Light day.

Week deliverable: The Enterprise Knowledge Assistant deployed, monitored, eval-gated, and documented.

Week 31: Portfolio polish

Goal: Make your work impossible to ignore.

Day	Focus	Do exactly this
Day 1	Pick	Your best 3 to 4 projects.
Day 2	READMEs	Problem, demo gif, architecture, how to run.
Day 3	Live demos	A working hosted demo for each.
Day 4	Write-ups	A short post per project (LinkedIn or a blog).
Day 5	Profile	Clean GitHub profile and pinned repos.
Day 6	Video	A 60-second demo video per project.
Day 7	Review	Light day.

Week deliverable: A portfolio of 3 to 4 deployed projects with strong READMEs and demos.

Week 32: Interview + job hunt

Goal: Convert the work into offers.

Day	Focus	Do exactly this
Day 1	Topics	Prompting, RAG, agents, eval, tradeoffs: the interview surface.
Day 2	System design	Design an LLM app: a RAG or agent system end to end.
Day 3	Tell the story	Explain your projects with the STAR structure.
Day 4	Resume	Lead with shipped AI projects and their impact.
Day 5	Apply	AI engineer / GenAI / LLM roles + open-source contributions.
Day 6	Mock	Do a mock interview. Fix the weak spots.
Day 7	Keep going	You are job-ready. Keep building and reading every week.

Week deliverable: A polished resume, 5+ applications out, and one mock interview done.

Part B: concept reference

Read each section in the week the plan points you to it. This is the 'understand it' half that makes you more than a tutorial-follower.

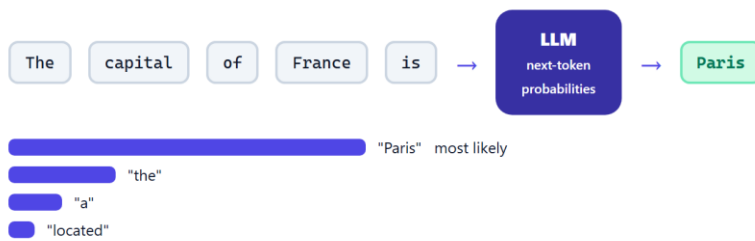
Beyond the basics: the rest of this sheet goes deeper than a fresher interview strictly requires. For interviews, master prompting, RAG, and one agent project, and be able to explain evaluation. The deployment, fine-tuning, and LLMops sections are recognise-the-words depth for freshers and become must-know once you are on the job.

How an LLM actually works

An LLM is a next-token predictor. Given the text so far, it outputs a probability for every possible next token, picks one, appends it, and repeats. That loop, run fast over a huge model, is all of it. It is not looking anything up in a database and it is not reasoning the way a person does.

How an LLM generates: one token at a time

It predicts the next token, appends it, and repeats. That loop is the whole thing.



Temperature decides how boldly it picks the top option. The picked token is added to the input and the model runs again for the next one.

One step of generation: context in, next token out, repeat.

Aha - Model ek super-autocomplete hai. Tum jo likhte ho uske baad sabse probable agla word guess karta hai, phir guess ko input mein jod ke fir se guess karta hai. Isiliye galat baat bhi confidently bol deta hai, kyunki use sirf 'kya sound karta hai sahi' pata hai, 'kya sach hai' nahi.

Term	What it means	Why you care
Token	A chunk of text (often part of a word)	You are billed per token; context is measured in tokens
Context window	Max tokens in + out per call	Inputs over the limit are rejected, not silently handled; RAG keeps you under it
Temperature	Randomness of the pick	Low = consistent, high = creative
Top-p / top-k	Limit the candidate tokens	Another way to control randomness
Pretraining	Learning language from huge text	Done once by the model maker, not you
Fine-tuning	Nudging a model on your data	You may do a little; it is a last resort
Inference	Running the model to get output	This is what your app does on every call
Hallucination	Confident wrong output	Reduce with grounding, citations, and a refusal path

The prompt engineering pattern bank

Prompting is the skill that pays off most in AI engineering. Most quality gains come from a better prompt, not a bigger model. These are the patterns worth memorizing.

Pattern	What it is	Use it when
Role / persona	Tell the model who it is in the system prompt	You want a consistent voice or expertise
Zero-shot	Just ask, no examples	The task is common and simple
Few-shot	Give 2-5 input/output examples	You need a specific format or style
Structured output	Demand JSON / a schema	Your code must parse the result

Pattern	What it is	Use it when
Delimiters	Wrap input in clear markers	User text could be confused with instructions
Chain-of-thought	Ask it to think step by step	Reasoning, math, multi-step logic
ReAct	Reason, then act with a tool, then observe	The model needs external info or actions
Decomposition	Split one big task into chained prompts	One prompt is doing too much and failing
Self-check	Ask the model to critique its own answer	Quality matters more than latency

The iteration loop that actually improves prompts: write it, run it on 10 to 20 real inputs, collect the failures, change ONE thing, re-run. Never tweak a prompt against a single example.

Structured generation: making output your code can trust

Free-text output is fine for a chat box and useless for a program. The moment another function depends on the model's answer, you need structured generation: the model returns data in a shape you defined, and you validate it before using it. This is one of the most common real interview topics because every production feature relies on it.

- JSON schema / structured outputs: tell the API the exact shape you want. Most providers can now guarantee valid JSON that matches a schema.
- Pydantic: define the shape as a Python class, and parse the model output into it. If the data is wrong, parsing fails loudly instead of silently feeding garbage downstream.
- Response validation: check types, ranges, and allowed values after parsing. A field that should be one of three statuses must actually be one of those three.
- Guarded generation: if validation fails, do not crash and do not trust it. Retry once, fall back to a simpler model or a default, or return a clean error. Decide this on purpose.

Aha - Model se kabhi kacha text le ke aage mat bhej. Pehle ek shape define karo (pydantic class), phir output ko us shape mein daalo. Agar fit nahi hua toh wahi pakad lo, taaki galat data chup-chaap tumhare app mein na ghus jaaye.

Embeddings and vector databases

An embedding turns text into a vector (a list of numbers) so that similar meanings land close together. Once your documents are vectors, finding the relevant ones for a question is just finding the nearest vectors. A vector database stores those vectors and does that search fast.

Aha - Socho har sentence ko ek map pe ek point bana diya. Milte-julte matlab waale points paas-paas, alag matlab waale door. Sawaal aaya toh use bhi point bana ke uske sabse paas waale points utha lo, wahi tumhare relevant documents hain.

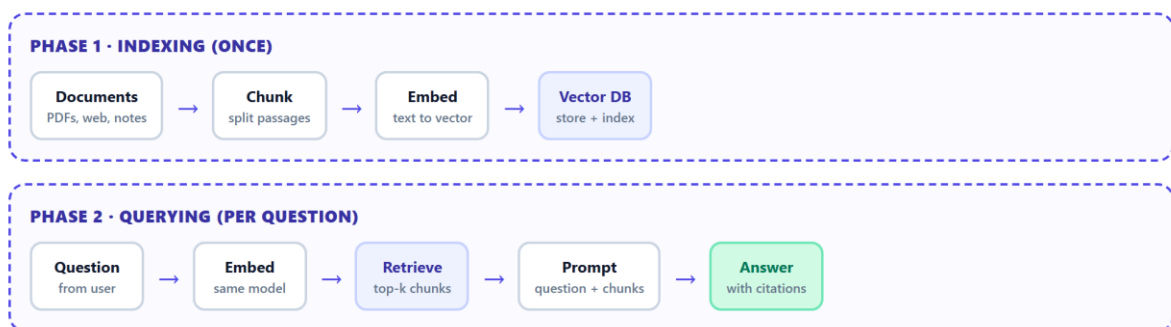
- **Chunking:** split big documents into passages before embedding. Too big and retrieval is vague; too small and it loses context. Start around 300 to 400 tokens per chunk with 20 to 50 tokens overlap, then measure retrieval quality on real questions and adjust from there.
- **Similarity:** cosine similarity measures the angle between two vectors. Closer angle, closer meaning.
- **Vector DB choices:** Chroma and FAISS for local/learning; pgvector if you already use Postgres; Pinecone, Qdrant, and Weaviate for managed scale. Learn one, the rest follow.
- **Hybrid search:** combine keyword search with vector search to catch exact terms (names, IDs) that pure semantic search can miss.
- **Debug retrieval first:** for one question, print the top chunks you actually retrieved and check the answer is really inside them. If it is not, the fault is retrieval (embedding model, chunk size, re-ranking), not the prompt. Fix that order before touching anything else.

RAG: the pattern that gets you hired

Retrieval-augmented generation is how you make a general model answer using your specific, private, or fresh data without retraining it. It is the most common real-world AI engineering pattern, so make your strongest project a RAG project.

The RAG pipeline

Index your documents once. Then retrieve and generate for every question.



The RAG pipeline: index your docs once, then retrieve + generate per question.

The pipeline has two phases:

- Indexing (once): load documents, split into chunks, embed each chunk, store the vectors in a vector DB with their source metadata.
- Querying (per question): embed the question, retrieve the most similar chunks, put them into the prompt as context, and ask the model to answer using only that context and cite the source.

Why RAG beats fine-tuning for knowledge: RAG updates instantly when you add a document, shows sources so users can trust it, and keeps private data out of the model. Fine-tuning bakes knowledge in, goes stale, and cannot cite. Reach for RAG first, almost always.

Common RAG failure modes and the fix:

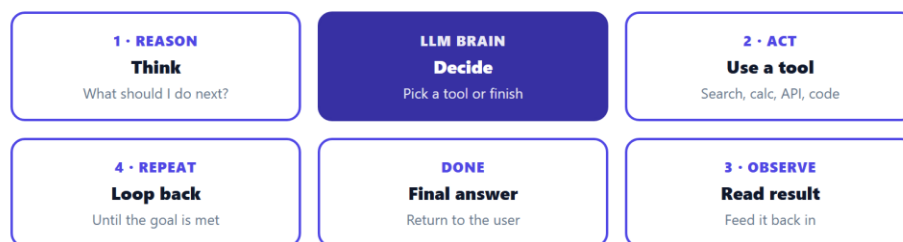
Symptom	Likely cause	Fix
Answer ignores the docs	Retrieved chunks are irrelevant	Better chunking, re-ranking, query rewriting
Right doc, wrong detail	Chunk too big or split mid-idea	Smaller chunks with overlap
Makes things up anyway	Prompt does not constrain to context	Instruct 'answer only from context, else say you do not know'
Misses exact names/IDs	Pure semantic search	Add keyword / hybrid search
Slow or expensive	Retrieving too many chunks	Retrieve fewer, re-rank the top ones

Agents: LLMs that take actions

An agent is an LLM in a loop with tools. Instead of just answering, it can decide to call a tool (search, calculator, code runner, your API), read the result, and decide the next step, until the task is done. The core loop is ReAct: reason, act, observe, repeat.

The agent loop (ReAct)

Reason, act with a tool, observe the result, repeat until done.



Always set a stop condition: max steps + a cost cap. Without it, an agent can loop forever and burn money.

The ReAct loop. A stop condition prevents infinite, costly loops.

Aha - Agent ek intern jaisa hai jise tum tools de dete ho. Wo sochta hai 'kya karna chahiye', ek tool use karta hai, result dekhta hai, phir agla kadam sochta hai. Tumhara kaam hai use

seedhe rails pe rakhna: max steps, allowed tools, aur galat hone pe rok.

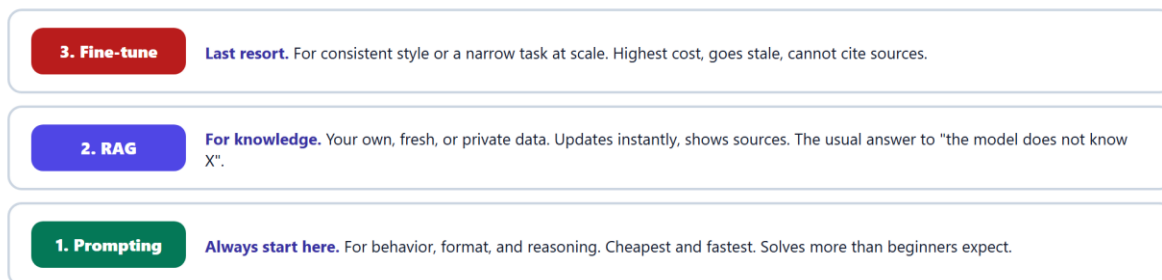
- Tools are the agent's hands. Each tool is a function with a clear name and description so the model knows when to use it.
- Always set a stop condition: a max number of steps and a cost cap. Agents loop, and loops cost money and time.
- Guardrails: validate tool inputs, allow-list what tools can do, and add human-in-the-loop for risky actions.
- Frameworks (LangGraph, smolagents, LlamaIndex, CrewAI, OpenAI Agents SDK) remove boilerplate. Build the loop by hand once first so you understand what they hide.
- MCP (Model Context Protocol) is an emerging standard for plugging tools and data sources into agents in a consistent way.

Prompt vs RAG vs fine-tune: the decision

Beginners reach for fine-tuning far too early. The right order is almost always prompt first, then add RAG, and only fine-tune if both fall short. This single decision shows interviewers you think like an engineer, not a hype-follower.

Prompt vs RAG vs fine-tune: climb in order

Most problems are solved on the bottom two rungs. Fine-tune only if both fall short.



Knowledge problem → RAG. Behavior problem → prompt, then fine-tune.

Climb the ladder in order. Most problems are solved on the first two rungs.

Lever	Best for	Cost / effort	Limitation
Prompting	Behavior, format, reasoning style	Lowest	Cannot add new knowledge
RAG	Your own / fresh / private knowledge	Medium	Needs a retrieval pipeline
Fine-tuning	Consistent style/format at scale, after prompting falls short	Highest	Goes stale, cannot cite, needs data

Rule of thumb: if the problem is 'the model does not KNOW X', use RAG. If it is 'the model does not BEHAVE the way I want', try prompting, then fine-tuning. Knowledge means RAG,

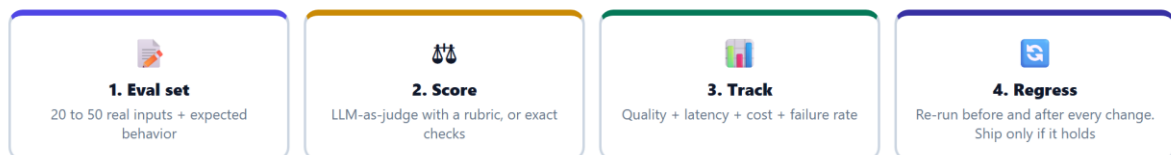
behavior means prompt or fine-tune.

Evaluation and observability

If you cannot measure quality, you cannot improve it and you cannot trust a change. Real AI engineers build an eval set early and re-run it on every change, the same way developers run tests. 'It looked fine when I tried it' is the beginner trap.

Evaluation turns vibes into a number

Build it early. Re-run it on every change, like tests.



Eval turns vibes into a number you can track and defend.

- Eval set: a fixed list of inputs paired with the expected behavior or a rubric. Start with 20 to 50 real examples.
- LLM-as-judge: use a model with a clear rubric to score outputs at scale, calibrated against a few human-scored examples.
- Track the operational metrics too: latency, cost per call, and failure/error rate.
- Observability tools (LangSmith, Langfuse, Phoenix) trace every step of a chain or agent so you can see where it went wrong.
- Regression test: before changing a prompt, run the eval; after, run it again. Ship only if the score holds or improves.

Agents need their own metrics. 'Did it sound right' is not enough when the thing took five steps and called three tools. Track these for any agent you ship:

Agent metric	What it measures	Why it matters
Task completion rate	Share of tasks finished correctly end to end	The headline number: does it actually work
Tool selection accuracy	How often it picks the right tool	A wrong tool choice derails the whole run
Hallucinated tool calls	Calls to tools/args that do not exist	A common, silent failure that crashes runs
Cost per task	Tokens and calls spent per completed task	Agents loop; cost is how you catch runaway loops
Latency	Time to finish a task	Multi-step agents get slow fast; users feel it

Production LLM app architecture

A toy notebook and a production app look very different. This is the shape of a real LLM app, and system-design interviews for AI roles ask you to draw exactly this.

Production LLM app architecture

A real app is mostly guardrails and plumbing around the model call.



A production LLM app: more guardrails and plumbing than model.

- A frontend or API gateway with auth and rate limiting.
- An input guard: validate and sanitize user input against prompt injection.
- Retrieval (RAG) and/or tool/agent orchestration in the middle.
- The model call itself, often with a cache and a fallback to a cheaper or backup model.
- An output guard: validation, moderation, and citation checks before the user sees anything.
- Logging, tracing, evals, and cost monitoring wrapped around all of it.

Safety and security

Every LLM product is attackable, and the attacks are unique to this field. You do not need to be a security researcher, but you must know these and add basic defenses, or your app leaks data and embarrasses you.

Threat	What it is	Defense
Prompt injection	Malicious input hijacks the instructions	Separate + sanitize user input, least-privilege tools
Jailbreak	Tricking the model past its rules	Output filtering, moderation, refusal prompts
Data leakage	Secrets or PII end up in prompts/logs	Redact PII, do not log raw prompts, scope keys
Hallucination	Confident false output	Ground with RAG, require citations, allow 'I do not know'
Excessive agency	An agent does something harmful	Allow-lists, human-in-the-loop, max steps

Prompt injection is the number one LLM vulnerability and has no perfect fix. Treat all user and retrieved text as untrusted, never give an agent more power than the task needs, and always validate what comes out before acting on it.

Prompt injection gets all the attention, but an LLM app is still a normal web service, so the boring security still applies. Enterprise interviewers expect you to know these:

Concern	What it covers	Baseline you should know
API abuse	Scraping, spam, and cost attacks on your endpoint	Per-user quotas and abuse detection
Rate limiting	Capping requests per user/key/time	A limit per API key and per IP
Auth	Who is allowed to call your service	API keys or OAuth; never an open endpoint
Secrets management	Where keys and credentials live	Env vars or a secret store, never in code or Git
RBAC	Role-based access: who can do what	Separate user, admin, and service roles
Tenant isolation	Keeping customer A's data away from B	Scope every query and retrieval by tenant

Cost engineering: the skill nobody teaches

On a real team, the bill is a feature. A working app that costs too much per request gets shut down. Cutting cost and latency without losing quality is a core part of the job and a frequent interview question, so learn the standard moves.

Technique	What it does	When to reach for it
Model routing	Send easy calls to a small/cheap model, hard ones to a big one	Most requests are easy; do not pay top price for all
Prompt compression	Trim the prompt: fewer examples, shorter context	Prompts have grown bloated with unused context
Caching	Reuse the response for an identical request	The same input recurs (FAQs, repeated questions)
Semantic caching	Reuse the answer for a similar (not identical) question	Many questions mean the same thing in different words
Batch processing	Group many requests into one job	Offline or bulk work that is not real-time
Token optimization	Shorter outputs, retrieve fewer chunks, cap max tokens	You are paying for tokens you do not use

Cheapest win first: log cost per request, find your most expensive calls, and route them to a smaller model or cache them. You usually cut a big chunk of the bill before touching anything clever. Note exact token prices change often, so always check the provider's live pricing page.

Multimodal AI: beyond text

More and more AI jobs need images, documents, and audio, not just text. You do not need to train vision models, but you should be able to use them. The good news: the patterns you already know (API calls, embeddings, RAG) extend straight to other media.

- Vision APIs: send an image to a multimodal model and ask questions about it (describe, classify, extract). Same API shape you already use, with an image added.
- Image embeddings: turn images into vectors so you can do similarity search over pictures, the same way you do over text.
- OCR pipelines: pull text out of scanned PDFs and photos so it can be chunked, embedded, and fed into RAG. This is how most 'chat with my documents' apps handle real-world files.
- Speech systems: speech-to-text (transcribe audio in) and text-to-speech (voice out). The base of voice assistants and meeting-notes tools.
- Multimodal RAG: retrieve across text and images together, so an answer can cite a paragraph and the diagram next to it.

Aha - Multimodal koi nayi duniya nahi hai. Wahi RAG aur API calls, bas input ab image ya audio bhi ho sakta hai. Photo ko bhi vector banao, search karo, model ko context do. Pattern same, media alag.

The data layer behind real AI apps

A vector DB is not the only database in a production AI app. Almost every real app also uses a normal database for users, sessions, and records, and a cache for speed. Interviewers notice fast when a candidate can build RAG but cannot model a simple users table.

Store	Role in an AI app	What to be able to do
Postgres (SQL)	Users, sessions, app records, audit logs	CRUD, joins, indexes, basic schema design
Redis	Caching, rate-limit counters, session state	Get/set with expiry, use it as a response cache
Vector DB	Embeddings for retrieval	Index, query top-k, metadata filters (Month 5)

One step up, production systems pass work around with queues and event systems. You do not need to build these yet, but recognize the names and what they are for:

- Kafka and RabbitMQ: message queues that move work between services so a slow LLM job does not block the request.
- Azure Service Bus and Azure Event Grid: managed messaging and event routing on Azure.

- The pattern to recognize: accept the request, drop the heavy LLM work on a queue, process it in the background, and notify when done. This is how real apps stay responsive.

Part C: career assets

The tools you'll meet (pick one per layer)

Do not learn all of these. Pick one per row, get good, and know the others exist. Tools churn fast; the concept is what transfers.

Layer	Popular options	Start with
Language	Python	Python
Closed model API	OpenAI, Anthropic Claude, Google Gemini	Whichever has a free tier for you
Open model	Llama, Mistral, Qwen, Gemma	Run one via Ollama
App UI	Streamlit, Gradio, Next.js	Streamlit or Gradio
Orchestration	LangChain, LlamaIndex, plain SDK	Plain SDK first, then LlamaIndex
Vector DB	Chroma, FAISS, pgvector, Pinecone, Qdrant	Chroma locally
Agents	LangGraph, smolagents, CrewAI, Agents SDK	smolagents or LangGraph
Eval / tracing	LangSmith, Langfuse, Phoenix	Langfuse (open source)
Deploy	FastAPI + Render/Railway, HF Spaces, Docker	HF Spaces, then FastAPI

AI-assisted development (the tools you'll use daily)

AI engineers build with AI coding tools, not just on top of models. Using these well is now an expected skill, and being fluent with them makes you noticeably faster. Learn to drive them, do not let them drive you: you still have to read and understand every line they produce.

Tool	What it is	Good for
GitHub Copilot	AI autocomplete + chat in your editor	Day-to-day coding, the easy default
Cursor	An editor built around an AI agent	Whole-file and multi-file edits with repo context
Claude Code	A terminal coding agent	Larger refactors and multi-step tasks
Windsurf	Agentic AI editor	Similar to Cursor; agent-driven edits
OpenAI Codex	Code-focused model / agent	Code generation and automation

- Prompt coding agents like a teammate: give the goal, the constraints, and the files that matter. Vague prompts get vague code.
- Repository context is what makes them good: point the tool at the right files so it understands your code instead of guessing.
- Agentic coding workflow: let the tool plan, edit across files, and run tests, but review every change. You own the code, not the tool.

- Never paste secrets or private code into a tool you do not trust, and always read what it wrote before you commit it.

The project ladder (your portfolio)

Your projects, not your certificates, get you hired. Build these in order. Each one proves a specific skill an interviewer is checking for.

#	Project	Proves you can
1	API-calling script (Month 1)	Code, use Git, work with JSON and APIs
2	Prompt-only tool (Month 3)	Engineer prompts for a real task
3	Deployed chat app (Month 4)	Build and ship a full LLM app with a UI
4	Chat-with-your-docs RAG (Month 5)	Build the pattern companies actually need
5	Task-doing agent (Month 6)	Build tool-using agents with guardrails
6	Fine-tuned + evaluated model (Month 7)	Measure quality and improve a model
7	Flagship: Enterprise Knowledge Assistant (Month 8)	Combine RAG, agents, eval, and deploy into one real product

Make the RAG project (4) and the flagship (7) your two showcase repos. Strong README (problem, demo gif, architecture diagram, how to run), a live hosted demo, and an honest 'what I would improve' section. That combination outperforms a wall of tutorials.

The flagship project: Enterprise Knowledge Assistant

One excellent project beats five half-built ones. Your Month 8 flagship is the build that ties the whole roadmap together and gives an interviewer a single, deep thing to dig into. The earlier capstones are how you learn each piece; this is where you assemble them.

What it pulls together:

- RAG: retrieval over a real document set, with citations.
- Agents: an agent that can use a few tools on top of retrieval.
- MCP: at least one tool exposed through a custom MCP server.
- Evaluation: a golden set with pass/fail checks that runs on every change.
- Monitoring: logging, tracing, and a cost view in production.
- Deployment: a containerized service behind a real API.
- Auth: protected endpoints, not an open URL.
- Caching: response and semantic caching to keep cost and latency down.

In interviews, this one project answers most questions on its own: architecture, retrieval choices, agent guardrails, how you evaluated it, what it costs, and what you would do next.

Depth on one real system reads as senior; breadth across toy demos reads as a tutorial list.

Build vs buy: when to use what

A senior-flavored interview question is 'what would you actually use here', and the trap is reaching for the fanciest option. Often the right answer is plain software, not AI at all. Being able to pick the simplest tool that solves the problem is exactly what they are testing.

Reach for	When the problem is	Do not use it when
Traditional software	Rules are fixed and known	You are forcing an LLM onto a plain if/else
Search	Users just need to find existing documents	The answer needs synthesis, not a link list
Prompting	You need behavior, format, or reasoning	The model lacks the knowledge entirely
RAG	The model needs your private or fresh data	The data fits in the prompt and rarely changes
Agent	The task needs several steps and tools	A single call would do; agents add cost and risk
Workflow	Steps are fixed and you want them reliable	You need open-ended decisions (use an agent)
Fine-tuning	Prompting + RAG both fell short at scale	You have not tried prompting and RAG first

The winning instinct: start with the simplest thing that could work and only climb up when it fails. Many candidates fail this by jumping straight to agents or fine-tuning for a problem that a search box or one prompt would solve.

Interview prep: AI engineer questions

The questions cluster into a few buckets. If you did the projects, you can answer all of these.

Concept questions

- How does an LLM generate text? What is a token and a context window?
- What is the difference between prompting, RAG, and fine-tuning, and when do you use each?
- Walk me through a RAG pipeline end to end. Where does it commonly fail?
- What is an embedding? How does semantic search work?
- What is an agent? What is the ReAct loop? How do you stop it looping forever?

- What is prompt injection and how do you defend against it?
- How do you evaluate an LLM app? What is LLM-as-judge?
- Temperature, top-p, top-k: what do they do?

System design questions

- Design a chatbot over a company's internal documents (this is RAG: draw indexing + querying).
- Design a PDF assistant: upload a document and ask questions about it (OCR, chunking, RAG).
- Design a coding assistant: answer questions about a repo (repo context, retrieval, tools).
- Design a customer-support agent that can look up orders and issue refunds (tools, guardrails, human-in-the-loop).
- How would you cut the cost and latency of an LLM feature in half? (caching, smaller models, fewer tokens, retrieval limits).
- How do you keep answers grounded and prevent hallucination in a production app?

Project questions (the ones that decide it)

- Walk me through your RAG project. Why those chunk sizes? How did you evaluate it?
- What broke, and how did you debug it?
- What would you do differently with more time?

Answer shape that lands: define the term in one line, give ONE concrete example from a project you built, then stop. 'A vector DB stores embeddings for fast similarity search. In my docs-chat project I used Chroma to retrieve the top 4 chunks per question.' That beats a textbook paragraph.

Common mistakes that stall beginners

- Tutorial hell: watching course after course and never shipping. Build after every concept.
- Skipping Python and fundamentals to rush to agents. You will hit a wall in week 2.
- Fine-tuning first. It is almost never the right first move; prompt and RAG first.
- Chasing every new framework. Pick one per layer and go deep.
- Never evaluating. 'It looked fine' is not a metric. Build an eval set early.
- Ignoring cost. Log token spend from day one and cap it.
- No deployed projects. A local notebook does not prove you can ship. Deploy everything.
- Copying tutorial repos without understanding. Interviewers ask 'why', and copies cannot answer.

The career reality check

Honest talk, because it saves heartbreak. Finishing this roadmap does not equal a job by itself. An AI engineer is still a software engineer, so companies expect the normal fundamentals on top of the AI skills. The roadmap makes you build-capable; these make you hireable.

- DSA: enough to clear a standard coding round. You cannot skip this for most roles.
- CS fundamentals: OS, networks, DBMS, and OOP basics still come up.
- Backend skills: APIs, databases, auth, and deployment, which this roadmap builds in Months 4 and 8.
- Resume: lead with shipped, deployed AI projects and their impact, not a list of courses.
- Communication: you must explain your projects and tradeoffs clearly. Interviews test this as much as code.

Run this roadmap alongside your DSA and core-subjects prep, not instead of it. The AI skills are the differentiator that makes you stand out; the fundamentals are the gate you still have to clear. You need both.

Your hiring timeline (run this in parallel)

Do not wait until the last week to think about the job. Getting hired is its own track that runs next to the build track. Start early so applications land when your portfolio is ready.

From around	Do this	Why now
Month 6	Start networking: post what you build, join AI communities, talk to people in roles	Relationships take months; start before you need them
Month 7	Post projects publicly: write-ups, demos, a clean GitHub	Your work needs to be visible before you apply
Month 8	Apply in volume: AI/GenAI/LLM roles, plus open-source contributions	Now the portfolio backs up the applications

Contributing to open source (how freshers stand out)

A merged pull request to a real AI project is proof you can work in a shared codebase, which a solo portfolio cannot show. It is one of the strongest signals a fresher can send, and the entry bar is lower than people think.

8. Start with a documentation fix: typos, unclear setup steps, a missing example. Low risk, real contribution, and you learn the project's workflow.

9. Read the issues: most repos label beginner-friendly issues as 'good first issue'. Pick one and ask before you start.
10. Target projects you already use: LangChain, LlamaIndex, and other open AI repos move fast and have plenty of approachable issues.
11. Build up: a docs fix, then a small bug, then a feature. Each merged PR is a line on your resume and a name maintainers recognize.

What to skip (do not waste months here)

Saying no is half the roadmap. These topics are real and interesting, but they are not what an AI engineer job needs, and students burn months on them before quitting. Skip them for now; come back later only if a specific role asks.

Skip for now	Why it is a trap for this path
Training LLMs from scratch	You will call pretrained models, not train them. Months of compute and math for nothing
Transformer math derivations	Intuition is enough; deriving attention by hand does not ship a single feature
Advanced calculus / heavy math	AI engineering needs light intuition, not a math degree
CNN / computer-vision research	Use vision APIs instead; research-depth CV is a different job
GANs and older generative models	Not what LLM-app roles use day to day
Every new framework that trends	Tools churn weekly; depth on one beats shallow on ten

The pattern: skip anything that is about building or researching models from the ground up. Your job is to turn existing models into products. Spend your hours on RAG, agents, evaluation, and shipping, which is exactly where the work actually is.

Curated resources (free first)

Start with the free, primary sources below. They cover the whole roadmap.

Resource: [roadmap.sh AI Engineer \(the canonical roadmap\)](#)

Resource: [Microsoft: Generative AI for Beginners \(21 lessons\)](#)

Resource: [Hugging Face LLM Course](#)

Resource: [Hugging Face AI Agents Course](#)

Resource: [Microsoft: AI Agents for Beginners](#)

Resource: [Microsoft: MCP for Beginners](#)

Resource: [OpenAI Prompt Engineering guide \(docs\)](#)

Resource: [Anthropic: Building effective agents](#)

Use these as your spine. For Python, any free beginner course works; the point is to build, not to collect courses. Treat tutorials as reference while you build the project for that week.

Final word

The AI engineer who gets hired is not the one who watched the most videos. It is the one who shipped four real, deployed projects and can explain every decision in them. Follow the chain, build in public, evaluate everything, and do not fine-tune before you have to. Eight months of this and you will not be 'learning AI' anymore. You will be building it.

Appendix: the jargon decoder

Every term on this sheet, in one line each.

Term	Plain meaning
LLM	Large Language Model: predicts the next token to generate text
Token	A chunk of text the model reads/writes; billing and context are in tokens
Context window	The maximum tokens (input + output) a model handles per call
Prompt	The full text you send the model: instructions, context, examples
System prompt	The standing instructions that set the model's role and rules
Few-shot	Putting example input/output pairs in the prompt to steer it
Chain-of-thought	Asking the model to reason step by step
Embedding	Text turned into a vector so similar meanings sit close together
Vector DB	A database that stores embeddings and finds the nearest ones fast
Chunking	Splitting documents into passages before embedding them
RAG	Retrieval-augmented generation: fetch your docs, then answer with them
Re-ranking	Reordering retrieved chunks so the most relevant are used
Tool / function calling	Letting the model trigger your code and use the result
Agent	An LLM in a loop that uses tools to complete a task
ReAct	Reason, act, observe: the core agent loop
MCP	Model Context Protocol: a standard way to plug tools/data into models
Fine-tuning	Training an existing model a little more on your own data
LoRA / PEFT	Cheap fine-tuning that updates only a small part of the model
SLM	Small Language Model: smaller, cheaper, faster, often good enough
Inference	Running a model to get an output (what your app does per call)
Temperature	How random the model's next-token pick is
Hallucination	A confident but wrong or made-up answer
Prompt injection	Malicious input that hijacks the model's instructions
Eval / LLM-as-judge	Scoring outputs with a dataset and/or a model + rubric
Observability	Tracing each step of a chain/agent to debug and monitor it
LLMOps	Running LLM apps in production: versioning, monitoring, CI/CD, cost

Term	Plain meaning
Grounding	Tying answers to real source text so they can be trusted/cited
Agentic AI	Systems where an LLM plans and acts in steps, not just answers
Function calling / tool use	The model asks your code to run and uses the result
Evals	The eval sets and scores you use to measure app quality
Synthetic data	Model-generated examples used to test or fine-tune
Context engineering	Deciding exactly what goes into the prompt's context, and what does not
AI gateway	A proxy in front of model APIs for keys, limits, caching, and logging
Agent orchestration	Coordinating steps, tools, and state across an agent run
Multi-agent system	Several agents with roles that collaborate on a task
Knowledge graph RAG	RAG that retrieves over a graph of entities and relations, not just text chunks
Semantic caching	Reusing an answer for a similar (not identical) question
Structured generation	Forcing model output into a defined, validated shape
Guardrails	Input/output checks that keep an app safe and on-task

Appendix: the Azure / OpenAI enterprise track

Many students target Microsoft and other enterprises, and a lot of enterprise AI runs on Azure. You do not need this to get started, but recognizing these services is a real employability boost for those roles. They are the managed, enterprise versions of patterns you already learn in this roadmap.

Azure service	What it is	Maps to (in this roadmap)
Azure OpenAI	OpenAI models hosted in Azure with enterprise controls	The model API layer (Month 4)
Azure AI Search	Managed search + vector retrieval	Vector DB + retrieval for RAG (Month 5)
Blob Storage	Object storage for files and documents	Where your source documents live
Document Intelligence	Extracts text and structure from PDFs and forms	The OCR step of a document RAG pipeline

Learn the open tools first (OpenAI/Anthropic APIs, Chroma, FastAPI). Once you understand the pattern, the Azure equivalents are a short hop, and naming them in an interview for a Microsoft or enterprise role shows you know where the field actually runs.